

Coding Convention:

Personal Finance and Budget Management System

Coding Convention Guide

Version 1.0

1. General Principles

- **Write clear, readable code** - Code is read more often than it's written
- **Be consistent** - Follow these conventions throughout the project
- **Comment when necessary** - Explain "why," not "what"
- **Keep it simple** - Follow the KISS principle from the design document

2. File Naming Conventions

- **Frontend (React)**
- **Components:** PascalCase - Dashboard.jsx, TransactionList.jsx
- **Utilities:** camelCase - formatCurrency.js, dateHelper.js
- **Styles:** Match component name - Dashboard.css
- **Backend (Node.js/Express)**
- **Controllers:** camelCase with suffix - userController.js, transactionController.js
- **Models:** PascalCase - User.js, Transaction.js, Category.js
- **Routes:** camelCase with suffix - userRoutes.js, transactionRoutes.js
- **Middleware:** camelCase with suffix - authMiddleware.js, errorHandler.js
- **General**
- Use lowercase with hyphens for folders: user-service/, transaction-controller/

3. Variable Naming

- **JavaScript (Frontend & Backend)**

// Variables and functions: camelCase

```
let userName = "John";
```

```
let totalExpense = 0;
```

```
function calculateTotal() { }
```

```
function getUserById() { }
```

// Constants: UPPER_SNAKE_CASE

```
const MAX_LOGIN_ATTEMPTS = 5;
```

```
const API_BASE_URL = "https://api.example.com";
```

```
// Classes: PascalCase
class TransactionService { }
class ReportGenerator { }

// Private variables: prefix with underscore
let _internalCache = {};

// Boolean variables: use is/has/can prefix
let isLoggedIn = false;
let hasPermission = true;
let canEdit = false;
```

4. Code Formatting

Indentation

- Use **2 spaces** for indentation (not tabs)
- Be consistent throughout the file

Line Length

- Keep lines under **100 characters** when possible
- Break long lines into multiple lines for readability
- **Spacing**

```
// Use spaces around operators
let total = income - expenses;

// Space after commas
function addTransaction(amount, category, date) { }

// No space inside parentheses
if (isValid) { } // Good
if ( isValid ) { } // Bad

// Blank line between logical sections
function processTransaction() {
  const data = fetchData();

  validateData(data);

  return saveData(data);
}
```

```
}
```

5. Comments

- **Single-line Comments**

```
// Calculate the total monthly expenses
```

```
let totalExpenses = calculateExpenses();
```

- **Multi-line Comments**

```
/**
```

```
 * Generates a financial report for the user
```

```
 * @param {string} userId - The user's ID
```

```
 * @param {string} startDate - Report start date
```

```
 * @param {string} endDate - Report end date
```

```
 * @returns {Object} The generated report
```

```
 */
```

```
function generateReport(userId, startDate, endDate) {
```

```
  // Implementation
```

```
}
```

- **When to Comment**

- Comment complex algorithms or business logic
- Explain "why" decisions were made, not "what" the code does
- Add TODO comments for future improvements: `// TODO: Add validation`

6. Function Guidelines

- **Function Length**

- Keep functions **short and focused** (ideally under 30 lines)
- One function should do one thing

- **Function Naming**

```
// Use verb + noun pattern
```

```
function getUser() { }
```

```
function createTransaction() { }
```

```
function deleteCategory() { }
```

```
function validateInput() { }
```

```
// Boolean functions: use is/has/can
```

```
function isValidEmail() { }
```

```
function hasPermission() { }
```

```
function canDelete() { }
```

- **Parameters**
- Limit to **3-4 parameters** when possible
- Use an object for more parameters:

// Bad

```
function createUser(name, email, password, age, country) { }
```

// Good

```
function createUser({ name, email, password, age, country }) { }
```

7. React Component Guidelines

- **Component Structure**

```
import React, { useState, useEffect } from 'react';
import './Dashboard.css';
```

// 1. Component definition

```
function Dashboard() {
```

// 2. State hooks

```
const [transactions, setTransactions] = useState([]);
```

// 3. Effect hooks

```
useEffect(() => {
  fetchTransactions();
}, []);
```

// 4. Helper functions

```
const fetchTransactions = async () => {
  // Implementation
};
```

// 5. Render

```
return (
  <div className="dashboard">
    {/* Component JSX */}
  </div>
);
}
```

export default Dashboard;

- **Props Naming**
- Use descriptive names: userName not un
- Boolean props: prefix with is/has/can: isActive, hasError

8. Error Handling

- **Frontend**

// Always use try-catch for async operations

```
async function fetchData() {  
  try {  
    const response = await fetch('/api/data');  
    const data = await response.json();  
    return data;  
  } catch (error) {  
    console.error('Error fetching data:', error);  
    // Show user-friendly message  
  }  
}
```

- **Backend**

// Use proper HTTP status codes

```
app.post('/api/transactions', async (req, res) => {  
  try {  
    const transaction = await createTransaction(req.body);  
    res.status(201).json(transaction);  
  } catch (error) {  
    console.error('Transaction creation failed:', error);  
    res.status(500).json({ error: 'Failed to create transaction' });  
  }  
});
```

9. Database & API Conventions

MongoDB Collection Names

- Use **plural, lowercase**: users, transactions, categories
- **API Endpoints**
- Use **RESTful conventions**:
 - GET /api/transactions - Get all transactions
 - GET /api/transactions/:id - Get one transaction

- POST /api/transactions - Create transaction
- PUT /api/transactions/:id - Update transaction
- DELETE /api/transactions/:id - Delete transaction

- **Request/Response Format**

// Request body: camelCase

```
{
  "userId": "123",
  "transactionAmount": 50.00,
  "categoryName": "Food"
}
```

// Response: camelCase with status

```
{
  "success": true,
  "data": { ... },
  "message": "Transaction created successfully"
}
```

10. Version Control (Git)

- **Commit Messages**

Use clear, descriptive commit messages:

Good:

- "Add user authentication middleware"
- "Fix transaction total calculation bug"
- "Update dashboard layout for mobile"

Bad:

- "Fixed stuff"
- "Update"
- "asdfgh"

- **Commit Message Format**

<type>: <description>

Types:

- feat: New feature
- fix: Bug fix
- docs: Documentation changes

- style: Code formatting (no logic change)
- refactor: Code restructuring
- test: Adding tests

Example:

feat: Add budget tracking feature

fix: Resolve login redirect issue

- **Branch Naming**
- main - Production-ready code
- develop - Development branch
- feature/feature-name - New features
- bugfix/bug-description - Bug fixes

11. Security Best Practices

// Never hardcode credentials

```
const API_KEY = process.env.API_KEY; // Good
```

```
const API_KEY = "abc123xyz"; // Bad
```

// Always validate user input

```
function createTransaction(data) {  
  if (!data.amount || data.amount <= 0) {  
    throw new Error('Invalid amount');  
  }  
  // Process transaction  
}
```

// Use HTTPS for API calls

// Hash passwords before storing

// Sanitize user inputs

12. Code Review Checklist

Before submitting code for review, ensure:

- [] Code follows naming conventions
- [] Functions are small and focused
- [] Comments explain complex logic
- [] Error handling is implemented
- [] No hardcoded values or credentials

- [] Code is properly formatted (2-space indentation)
- [] Variable names are descriptive
- [] Commit messages are clear

- **Quick Reference**

Item	Convention	Example
Variables	camelCase	userName, totalAmount
Constants	UPPER_SNAKE_CASE	MAX_RETRIES
Functions	camelCase (verb+noun)	getUser(), calculateTotal()
Classes	PascalCase	TransactionService
Files (React)	PascalCase	Dashboard.jsx
Files (Node)	camelCase	userController.js
API Endpoints	lowercase, plural	/api/transactions